Git & GitHub

Complete Training Notes — Day 1 to Day 4

Version Control for DevOps • Target: 10–12 LPA

Varun • All 4 Days Complete

Git Training — Progress Tracker

Day	Topic	Status
Git Day 1	Intro, init, add, commit, log, .gitignore	✅ Complete
Git Day 2	Branches, merging, conflict resolution	✅ Complete
Git Day 3	GitHub, remote, push, pull, clone, PRs	✅ Complete
Git Day 4	Stash, rebase, cherry-pick, reset, revert, reflog, tags	✅ Complete

GIT DAY 1 Introduction, Core Workflow & .gitignore

Git vs GitHub

Git	GitHub
Runs locally on your machine	Cloud platform — website
Tracks changes in files	Hosts repositories online
Created by Linus Torvalds 2005	Created 2008 — acquired by Microsoft
Works without internet	Requires internet
Free and open source	Free tier + paid plans

ℹ **Other platforms: GitLab (self-hostable, popular in enterprises), Bitbucket (Atlassian — corporate). All use Git underneath.**

Installing and Configuring Git

```
sudo apt install -y git      # install on Ubuntu
git --version                # verify — should show version number
```

```
# Configure — do this FIRST before anything else:
git config --global user.name "Varun"
git config --global user.email "varun@email.com"
git config --global core.editor "vim"
git config --global init.defaultBranch "main"

git config --list           # verify all settings
```

The Three Areas of Git — Most Important Concept

```
Working Directory  ──git add──>  Staging Area  ──git commit──>  Repository
(edit files here)               (prepare here)                  (saved forever)

# Memory trick — packing for a trip:
# Working Dir  = your room with all your stuff
# Staging Area = your suitcase (YOU choose what to pack)
# Repository   = suitcase closed, tagged — history saved permanently
```

> **You can change 10 files but only commit 3 of them together. Staging area gives you that precise control.**

Core Git Commands

```
git init                        # initialize new repository

git status                      # see current state — RUN THIS CONSTANTLY

# Staging files:
git add filename.txt            # stage one file
git add .                       # stage ALL changes
git add -p                      # interactive — choose specific parts

# Committing:
git commit -m "Add deployment script"
git commit -am "Fix typo"       # stage tracked + commit in one step

# Viewing history:
git log --oneline               # clean — one line per commit
git log --oneline --graph       # shows branch structure
git log --oneline -10           # last 10 commits only
git log --author="Varun"        # commits by specific person

# Seeing changes:
git diff                        # unstaged changes
git diff --staged               # staged not yet committed
```

Good vs Bad Commit Messages

Bad Message	Good Message
"fix"	"Fix disk space alert threshold in monitoring script"
"changes"	"Add nginx config for production load balancer"
"update"	"Update DB backup schedule to run at 1am daily"

"WIP"	"Remove hardcoded password — read from environment"
"asdfgh"	"Refactor deploy script to support multiple envs"

Format: Start with a verb. Be specific. Under 72 characters. Explain WHAT and WHY. Interviewers ask to see Git history — bad messages show unprofessional habits.

The .gitignore File

```
# Create BEFORE first commit:
vim .gitignore

# Always include:
.env                    # secrets — NEVER commit passwords
*.pem  *.key            # private keys
*.log                   # log files
node_modules/           # dependencies — too large, regenerated
__pycache__/  *.pyc     # Python cache
dist/  build/           # build output
.DS_Store               # Mac OS files
.vscode/  .idea/        # IDE files
*.tfstate               # Terraform state — has secrets
.terraform/             # Terraform directory

# Global .gitignore — applies to ALL your repos:
git config --global core.excludesfile ~/.gitignore_global
```

.env files contain passwords and API keys. If pushed to public GitHub they are exposed to the entire internet INSTANTLY — even if you delete the file later, it stays in Git history forever.

Complete First Workflow

```
git init
vim .gitignore                  # create FIRST before adding anything
git add .gitignore
git commit -m "Add gitignore for secrets and OS files"

git add deploy.sh nginx.conf
git status                      # verify what is staged
git commit -m "Add deployment script and nginx config"

git log --oneline               # verify history
git status                      # should show nothing to commit
```

Day 1 — Interview Questions

Q: What is Git and why is it used?

A: Git is a distributed version control system tracking every change to files over time. Used to collaborate, maintain history, revert to previous versions and work on features in parallel. Every DevOps team uses Git.

Q: What are the three areas of Git?

A: Working Directory where you edit files. Staging Area where you prepare changes with git add. Repository where history is permanently saved with git commit.

Q: What is the difference between git add . and git add -p?

A: git add . stages all changed files. git add -p goes through each change interactively letting you choose which parts to stage — useful for partial commits.

Q: Why should .env files never be committed to Git?

A: They contain secrets — passwords, API keys. Anyone with repo access reads them. On public repos this is an immediate security incident. Always add .env to .gitignore before first commit.

Q: What is a commit SHA?

A: A unique 40-character hash auto-generated for every commit. Used to reference specific commits — only first 7 characters needed.

GIT DAY 2 Branches, Merging & Conflict Resolution

What is a Branch?

A branch is an independent line of development — a pointer to a specific commit. Multiple developers work on different features simultaneously without affecting each other. Each works in isolation and merges when ready.

```
main branch:    A --- B --- C
                             \
feature branch:               D --- E --- F

# Main continues. Feature has its own commits.
# When feature done — merge F back into main.
```

Branch Commands

```
git branch                      # list all local branches (* = current)
git branch -a                   # list all including remote
git branch -v                   # list with last commit message

git branch feature-login        # create new branch
git switch feature-login        # switch to branch
git switch -c feature-login     # CREATE and switch — most common
git switch main                 # switch back to main

git branch --merged             # branches already merged
git branch --no-merged          # branches not yet merged

git branch -d feature-login     # delete merged branch — safe
git branch -D feature-login     # force delete — even if not merged
```

⚠ git switch vs git checkout: git switch is modern (Git 2.23+) — only for switching branches. git checkout is older and does multiple things. Use git switch for clarity.

What is HEAD?

HEAD is a pointer showing where you currently are — which branch and commit. When you switch branches HEAD moves. When you commit HEAD moves forward.

```
cat .git/HEAD
# ref: refs/heads/main   ← you are on main branch
```

Two Types of Merges

Fast-Forward Merge

```
# Happens when main has NOT moved since you branched:
Before:  main: A---B
                    \
         feature:    C---D

After fast-forward:
         main: A---B---C---D

# Git just moves the pointer — no merge commit created
git switch main
git merge feature-monitoring
# Output: Fast-forward
```

Three-Way Merge

```
# Happens when BOTH branches have new commits:
Before:  main: A---B---C
                    \
         feature:    D---E

After three-way:
         main: A---B---C---M
                    \     /
         feature:    D---E

# M is the merge commit — has two parents
git switch main
git merge feature-login
# Output: Merge made by recursive strategy
```

Merge Conflicts — Most Important Part

A conflict happens when BOTH branches changed the same part of the same file. Git cannot decide which version to keep — it asks you.

```
# What a conflict looks like inside the file:
server_name = "myapp"
<<<<<<< HEAD
port = 8080
=======
port = 9090
>>>>>>> feature-branch
database_host = "localhost"

# <<<<<<< HEAD     = YOUR version (current branch)
# =======          = divider
# >>>>>>> feature  = THEIR version (branch being merged)
```

Resolving Step by Step

```
# Step 1 — open conflicting file:
vim config.yml

# Step 2 — edit to keep what you want (remove ALL conflict markers):
server_name = "myapp"
```

```
port = 9090
database host = "localhost"

# Step 3 — stage the resolved file:
git add config.yml

# Step 4 — complete the merge:
git commit -m "Merge feature-branch — use port 9090"

# If conflicts too complex — abort and start over:
git merge --abort
```

Real DevOps Branching Strategies

GitHub Flow — Most Common for DevOps

```
# main = always deployable — production code
# all new work on feature branches

git switch -c feature/add-monitoring
# ... do work, make commits ...
git switch main
git merge feature/add-monitoring
git branch -d feature/add-monitoring
git push origin main                  # deploy to production
```

Branch Naming Conventions

Type	Format	Example
Feature	feature/description	feature/add-monitoring
Bug fix	bugfix/description	bugfix/fix-login-error
Hotfix	hotfix/description	hotfix/critical-security-patch
Release	release/version	release/v1.2.0

Day 2 — Interview Questions

Q: What is a Git branch?

A: A branch is an independent line of development — a pointer to a specific commit. Used so multiple developers work on different features without affecting each other.

Q: What is the difference between fast-forward and three-way merge?

A: Fast-forward happens when target branch has not moved — Git simply moves the pointer, no merge commit, clean linear history. Three-way happens when both branches have new commits — creates a merge commit combining both histories.

Q: What is a merge conflict and how do you resolve it?

A: Conflict occurs when both branches changed the same part of the same file. Git marks with conflict markers. Resolve by opening file, removing markers, keeping correct version, git add, git commit.

Q: What is the difference between git branch -d and -D?

A: -d safely deletes only if branch is fully merged. -D force deletes regardless. Always use -d first.

Q: Explain GitHub Flow.

A: Main always contains deployable production code. All work on feature branches. When done merge to main and delete branch. Simple and used by most modern DevOps teams.

GIT DAY 3 GitHub, Remote, Push, Pull & Pull Requests

Local vs Remote

```
LOCAL (your machine)                REMOTE (GitHub)

git_practice/                       github.com/varun/git_practice
├── .git/                           ├── main branch
└── deploy.sh                       └── pull requests

git push  ──────────────────────>   send commits up
git pull  <──────────────────────   bring commits down
```

Setting Up SSH with GitHub

```
# Generate key:
ssh-keygen -t ed25519 -C "varun@email.com"

# Copy public key:
cat ~/.ssh/id_ed25519.pub
# Go to GitHub → Settings → SSH Keys → New SSH Key → paste

# Test connection:
ssh -T git@github.com
# Output: Hi varun! You've successfully authenticated
```

Connecting Local Repo to GitHub

```
# Add remote — named origin by convention:
git remote add origin git@github.com:varun/devops-scripts.git

# Verify:
git remote -v
# origin  git@github.com:varun/devops-scripts.git (fetch)
# origin  git@github.com:varun/devops-scripts.git (push)

# Other remote commands:
git remote remove origin
git remote rename origin upstream
git remote set-url origin NEW_URL
```

git push — Sending to GitHub

```
git push -u origin main          # FIRST push — sets upstream tracking
git push                         # after -u set — just this
git push origin feature-login    # push specific branch
git push origin --all            # push ALL branches
git push origin --tags           # push all tags

# Force push (use with extreme caution):
```

```
git push --force-with-lease      # safer — checks for others' changes first
```

NEVER force push to main or any shared branch. It rewrites history and breaks everyone else who has pulled that branch. Only acceptable on your own feature branches after rebasing.

git clone — Downloading a Repository

```
git clone git@github.com:varun/devops-scripts.git
git clone git@github.com:varun/devops-scripts.git my-folder  # custom name
git clone --depth 1 git@github.com:varun/project.git         # shallow — latest
only

# clone = init + remote add + pull all — in one command
# origin is set automatically
```

git fetch vs git pull

git fetch	git pull
Downloads changes only	Downloads AND merges
Does not touch your files	Updates your branch immediately
Safe — inspect before merging	Faster — one step
Then: git merge origin/main	Equivalent to fetch + merge

```
# fetch — safe inspection workflow:
git fetch origin
git log --oneline HEAD..origin/main    # see what changed
git diff HEAD origin/main              # see exact differences
git merge origin/main                  # merge when confident

# pull — quick update:
git pull                               # fetch + merge
git pull --rebase                      # fetch + rebase (cleaner history)
```

Pull Requests — Heart of Collaboration

A PR is a request to merge your branch into another branch with a code review step in between. Team reviews code, CI runs tests, approved then merged.

Creating a PR — Step by Step

```
# Step 1 — push feature branch:
git switch -c feature/add-backup-script
echo "tar -czf /backup/home_$(date +%Y%m%d).tar.gz /home/varun/" > backup.sh
git add backup.sh
git commit -m "Add automated home directory backup script"
git push -u origin feature/add-backup-script

# Step 2 — on GitHub:
# Click "Compare & pull request"
# Add clear title and description
# Click "Create pull request"

# Step 3 — after review and approval:
```

```
# Click "Merge pull request"
# Click "Delete branch"

# Step 4 — locally clean up:
git switch main
git pull                                # get the merged changes
git branch -d feature/add-backup-script
```

Good PR Description Template

```
## What this does
Adds automated backup script creating daily tar.gz of home directory.

## Why
Currently no automated backups exist. Prevents data loss.

## How to test
Run ./backup.sh manually — verify .tar.gz appears in /backup/

## Related
Closes #45 — feature request for automated backups
```

Forking — Contributing to Other Projects

```
# 1. Fork on GitHub — click Fork button

# 2. Clone YOUR fork:
git clone git@github.com:varun/project.git

# 3. Add original as upstream:
git remote add upstream git@github.com:company/project.git

# 4. Verify remotes:
git remote -v
# origin    → your fork (push here)
# upstream  → original repo (pull updates from here)

# 5. Keep fork updated:
git fetch upstream
git merge upstream/main
git push origin main
```

Common GitHub Situations

```
# Push rejected — remote has changes you don't have:
git pull --rebase origin main    # bring in remote changes first
git push origin main             # now push succeeds

# Committed to wrong branch:
git log --oneline               # find commit SHA
git switch -c feature/my-change # create correct branch
git switch main
git reset HEAD~1                # remove from main (keep changes)

# Update PR with new changes after review feedback:
git switch feature/my-feature
# make requested changes
git add . && git commit -m "Address review — add error handling"
```

```
git push origin feature/my-feature  # PR updates automatically
```

Day 3 — Interview Questions

Q: What is the difference between git fetch and git pull?

A: git fetch downloads changes without merging — you inspect first. git pull does both fetch and merge. fetch is safer for production workflows.

Q: What is a Pull Request?

A: A request to merge one branch into another with a code review step. Team reviews, CI runs tests, approved then merged. Core to professional team workflows.

Q: What is the difference between origin and upstream?

A: origin is your own remote — where you push changes. upstream is the original repo you forked from — where you pull updates. Pull from upstream, push to origin, PR from origin to upstream.

Q: What does git clone --depth 1 do?

A: Creates a shallow clone — only latest commit without full history. Much faster. Used in CI/CD pipelines where only current code is needed.

Q: Your push is rejected because remote has changes. What do you do?

A: git pull --rebase origin main to bring in remote changes and replay your commits on top, then git push. --rebase keeps history clean.

GIT DAY 4 Advanced Git — Stash, Rebase, Cherry-Pick, Reset, Tags

git stash — Save Work Without Committing

Temporarily saves uncommitted changes so you can switch context — like a production hotfix interrupting feature work.

```
git stash                           # save all uncommitted changes
git stash push -m "half-done login"  # save with description
git stash list                      # see all stashes
git stash pop                       # restore latest AND remove from list
git stash apply                     # restore latest BUT keep in list
git stash apply stash@{2}           # restore specific stash
git stash drop stash@{0}            # delete specific stash
git stash clear                     # delete ALL stashes
git stash -u                        # also stash untracked new files
```

pop = restore and remove (use when done). apply = restore and keep (use when applying same changes to multiple branches).

⚠ Stashes are LOCAL only — not pushed to GitHub. Do not rely on stash for long-term storage. For temporary context switching only.

Real Scenario — Interrupted Work

```
# Working on feature — manager calls — production down!
git stash push -m "deploy script — half done"
git switch main
git switch -c hotfix/production-crash
# fix the issue
git commit -m "Fix auth crash on missing token"
git push origin hotfix/production-crash
```

```
# Production fixed — back to your work:
git switch feature/deploy-improvements
git stash pop          # half-done work is back
```

git rebase — Rewrite History Cleanly

```
# The problem with merge — cluttered history:
feature:  A---B---C---M---D---E---M---F
                 |           |
main:     A---B---C---G---H---I

# Rebase replays your commits on top of main:
Before:  main: A---B---C---D---E
                       \
         feature:       F---G

After git rebase main:
         main: A---B---C---D---E
                                \
         feature:                F'---G'

# Clean linear history — no merge commits
# Basic rebase:
git switch feature/my-feature
git rebase main

# If conflict during rebase:
vim conflicting-file.txt        # fix conflict
git add conflicting-file.txt
git rebase --continue
# or give up:
git rebase --abort
```

NEVER rebase commits that exist on a shared/public branch. Rebase rewrites SHAs — anyone who pulled old SHAs gets history conflicts. Only rebase your own local feature branches.

Interactive Rebase — Clean Up Before PR

```
git rebase -i HEAD~3    # edit last 3 commits

# Editor opens showing:
pick a1b2c3d Add monitoring script
pick d4e5f6g Fix typo
pick h7i8j9k Another typo fix

# Change pick to:
# pick    = keep as-is
# reword  = keep but edit message
# squash  = combine with previous commit
# fixup   = combine — discard this message
# drop    = delete this commit entirely

# Squash into one clean commit:
pick a1b2c3d Add monitoring script
squash d4e5f6g Fix typo
squash h7i8j9k Another typo fix
# Save — Git combines into one with a clean message
```

Merge	Rebase
Preserves complete history	Creates clean linear history
Safe on shared branches	NEVER on shared branches
Creates merge commits	No merge commits
Non-destructive	Rewrites commit SHAs

git cherry-pick — Take Specific Commits

```
# Apply one specific commit to current branch:
git cherry-pick a1b2c3d

# Apply multiple commits:
git cherry-pick a1b2c3d d4e5f6g

# Apply range:
git cherry-pick a1b2c3d..h7i8j9k

# Apply but do not commit yet:
git cherry-pick -n a1b2c3d

# If conflict:
git cherry-pick --continue
git cherry-pick --abort
```

Real Scenario — Backporting a Security Fix

```
# Critical fix on main — need it on old release branch too
git log --oneline main
# a1b2c3d Fix SQL injection in user auth  ← need this

git switch release-v1
git cherry-pick a1b2c3d    # apply ONLY this fix
git push origin release-v1 # deploy fix to old version
```

git reset — Undo Commits

Flag	Commit	Staging Area	Working Dir
--soft	Undone	Changes kept staged	Unchanged
--mixed (default)	Undone	Cleared	Changes kept
--hard	Undone	Cleared	Changes DELETED

```
git reset --soft HEAD~1     # undo commit — keep changes staged
git reset HEAD~1            # undo commit — unstage changes (default)
git reset --hard HEAD~1     # undo commit — DELETE all changes

# Reset to specific commit:
git reset --hard d4e5f6g    # everything after d4e5f6g is gone
```

⚠ git reset --hard deletes your changes permanently. Only use on local commits not yet pushed. For shared branches use git revert instead.

git revert — Safe Undo for Shared Branches

```
# Creates NEW commit that undoes previous commit:
Before:  A---B---C---D   (D is the bad commit)
After:   A---B---C---D---D'
#        D' undoes what D did — D still in history

git revert HEAD              # revert last commit
git revert a1b2c3d           # revert specific commit
git push origin main         # safe — history preserved
```

Use reset when	Use revert when
NOT pushed yet — local only	Already pushed to shared branch
You want to completely remove	You need to preserve history
Working alone on a branch	Others have pulled this branch

Simple rule: Reset locally. Revert remotely.

git reflog — Recover Anything

```
git reflog              # every HEAD movement ever recorded

# Output:
a1b2c3d HEAD@{0}: commit: Add monitoring
d4e5f6g HEAD@{1}: reset: moving to HEAD~1   ← accidental reset here
h7i8j9k HEAD@{2}: commit: Previous good state

# Recover lost commits after git reset --hard:
git reflog                          # find SHA of lost commit
git reset --hard d4e5f6g            # restore to that point
# or recover on new branch:
git switch -c recovery-branch d4e5f6g
```

Reflog entries expire after 90 days. You have 90 days to recover anything. Git is almost always recoverable if you act quickly.

git tag — Marking Releases

```
# Lightweight tag — just a pointer:
git tag v1.0.0

# Annotated tag — with metadata — PREFERRED for releases:
git tag -a v1.0.0 -m "Release v1.0.0 — initial production release"

git tag                              # list all tags
git tag -l "v1.*"                    # list tags matching pattern
git show v1.0.0                      # show tag details
git push origin v1.0.0               # push specific tag
git push origin --tags               # push all tags
git tag -d v1.0.0                    # delete locally
git push origin --delete v1.0.0      # delete on remote
```

Version	Change type	Example
v1.0.0 → v1.0.1	Patch — bug fixes only	Fix login crash

v1.0.0 → v1.1.0	Minor — new features	Add monitoring dashboard
v1.0.0 → v2.0.0	Major — breaking changes	Rewrite authentication

Tags trigger CI/CD pipelines in most setups. Push a v* tag → pipeline automatically builds Docker image → deploys to production.

Git Hooks — Automation at Every Step

```
ls .git/hooks/
# pre-commit     — runs before every commit
# commit-msg     — validates commit message
# pre-push       — runs before push
# post-merge     — runs after merge

# Example — enforce commit message format:
vim .git/hooks/commit-msg
#!/bin/bash
pattern="^(feat|fix|docs|refactor|test|chore): .+"
if ! grep -qE "$pattern" "$1"; then
    echo "ERROR: Use format: feat: description"
    exit 1
fi
chmod +x .git/hooks/commit-msg

# Example — run tests before push:
vim .git/hooks/pre-push
#!/bin/bash
./run_tests.sh || { echo "Tests failed — push blocked"; exit 1; }
chmod +x .git/hooks/pre-push
```

git bisect — Find the Breaking Commit

```
git bisect start
git bisect bad                    # current commit is broken
git bisect good v1.0.0            # this version was working

# Git checks out midpoint — you test:
git bisect good                   # this commit works fine
# or
git bisect bad                    # this commit is broken

# Repeat until Git finds exact bad commit
# Output: a1b2c3d is the first bad commit

git bisect reset                  # exit — back to original branch

# Automated with test script:
git bisect run ./test.sh          # Git runs test on each commit automatically
```

Day 4 — Interview Questions

Q: What is git stash?

A: Temporarily saves uncommitted changes so you can switch branches without committing incomplete work. git stash pop restores when you return.

Q: What is the difference between git merge and git rebase?

A: Merge creates a merge commit preserving full history — safe and non-destructive. Rebase replays commits on top creating clean linear history but rewrites SHAs. Never rebase shared branches.

Q: What is interactive rebase used for?

A: git rebase -i HEAD~N lets you edit, squash or drop recent commits. Used to clean up messy history before opening a PR — combine multiple small commits into one clean commit.

Q: What is git cherry-pick?

A: Applies a specific commit from one branch onto another without merging everything. Used for backporting — applying a critical fix to an older release branch.

Q: What is the difference between git reset and git revert?

A: reset moves branch pointer backwards rewriting history — local only. revert creates a new commit undoing changes preserving history — safe for shared branches. Reset locally, revert remotely.

Q: How do you recover commits after git reset --hard?

A: git reflog records every HEAD movement. Find SHA of lost commit in reflog then git reset --hard that_SHA to restore, or git switch -c recovery that_SHA.

Q: What are Git tags?

A: Permanent markers on specific commits — used for releases. Annotated tags include metadata. Use semantic versioning: major.minor.patch. Tags trigger CI/CD pipelines.

Q: What are Git hooks?

A: Scripts in .git/hooks/ that run automatically on git events. pre-commit, commit-msg, pre-push etc. Used to enforce standards, run tests and prevent bad commits locally.

MASTER REFERENCE All Git Commands — Quick Reference

Git Day 1 — Setup and Basics

Command	What it does
`git config --global user.name "Name"`	Set your identity
`git init`	Initialize new repository
`git status`	See current state — use constantly
`git add filename`	Stage specific file
`git add .`	Stage all changes
`git add -p`	Interactive staging
`git commit -m "message"`	Save staged changes to history
`git commit -am "message"`	Stage tracked files and commit
`git log --oneline`	View compact commit history
`git log --oneline --graph`	Visual branch history
`git diff`	Unstaged changes
`git diff --staged`	Staged changes not yet committed

Git Day 2 — Branches and Merging

Command	What it does
`git branch`	List local branches
`git branch name`	Create new branch
`git switch name`	Switch to branch
`git switch -c name`	Create AND switch in one step
`git merge branch`	Merge branch into current
`git merge --abort`	Cancel merge in progress
`git branch -d name`	Delete merged branch safely
`git branch -D name`	Force delete branch
`git log --graph --all`	See all branches visually
`git branch --merged`	Show merged branches
`git branch --no-merged`	Show unmerged branches

Git Day 3 — GitHub and Remotes

Command	What it does
`git remote add origin URL`	Connect local to GitHub
`git remote -v`	Show all remotes
`git push -u origin main`	Push and set upstream tracking
`git push`	Push current branch (after -u)
`git push origin --all`	Push all branches
`git push origin --tags`	Push all tags
`git clone URL`	Download entire repository
`git clone --depth 1 URL`	Shallow clone — latest only
`git fetch origin`	Download changes without merging
`git pull`	Fetch and merge
`git pull --rebase`	Fetch and rebase — cleaner
`git branch -a`	Show all branches including remote
`git remote add upstream URL`	Add original repo for forks

Git Day 4 — Advanced Commands

Command	What it does
`git stash`	Save uncommitted changes
`git stash push -m "desc"`	Save with description
`git stash pop`	Restore latest and remove

`git stash list`	See all stashes
`git rebase main`	Rebase onto main cleanly
`git rebase -i HEAD~3`	Interactive — edit last 3 commits
`git rebase --abort`	Cancel rebase
`git cherry-pick SHA`	Apply specific commit
`git reset --soft HEAD~1`	Undo commit — keep staged
`git reset HEAD~1`	Undo commit — keep in workdir
`git reset --hard HEAD~1`	Undo commit — delete changes
`git revert HEAD`	Safe undo — creates new commit
`git reflog`	See all HEAD movements
`git tag -a v1.0.0 -m "msg"`	Create annotated tag
`git push origin --tags`	Push all tags
`git bisect start/good/bad`	Binary search for bad commit

Git training COMPLETE — 4 days, all concepts covered for 10-12 LPA DevOps interviews!

Next in your journey: Docker → AWS → CI/CD → Kubernetes